Clustering and Differential Analysis

Yongjin Park

05 March 2026

Roadmap: From biology to methodology

14: Biological Networks

- Protein-protein, genetic interactions
- Cell-cell interaction networks
- Network as a tool: propagation

15: Clustering

- Graph-based clustering
- Single-cell example
- When clustering goes wrong

Today's lecture

1. Graph-based clustering
2. Single-cell clustering in practice
3. When clustering goes wrong
4. Discussion: How should we test cell-cell communication?
5. Appendix

Last time: networks encode relationships

- We learned how to build and visualize networks
- Protein-protein interactions, genetic interactions, cell-cell communication
- Today: how do we find **groups** within a network?

Recap: `igraph` in R

```r
library(igraph)
```

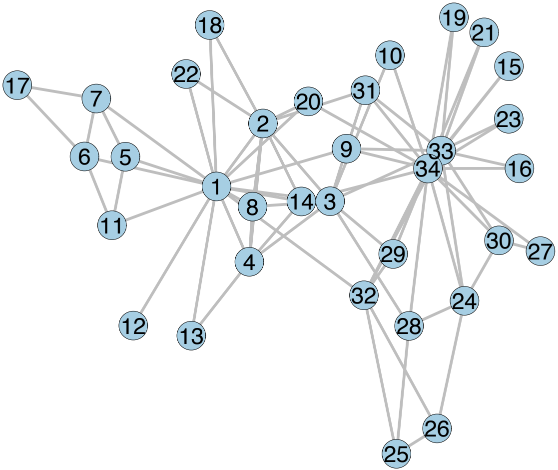
```r
karate <- .read.online("http://www-personal.umich.edu/~mejn/netdata/karate.zip")
karate
```

```
## IGRAPH cd4b8ea U--- 34 78 --
## + attr: id (v/n)
## + edges from cd4b8ea:
##  [1]  1-- 2  1-- 3  2-- 3  1-- 4  2-- 4  3-- 4  1-- 5  1-- 6  1-- 7  5-- 7
## [11]  6-- 7  1-- 8  2-- 8  3-- 8  4-- 8  1-- 9  3-- 9  3--10  1--11  5--11
## [21]  6--11  1--12  1--13  4--13  1--14  2--14  3--14  4--14  6--17  7--17
## [31]  1--18  2--18  1--20  2--20  1--22  2--22 24--26 25--26  3--28 24--28
## [41] 25--28  3--29 24--30 27--30  2--31  9--31  1--32 25--32 26--32 29--32
## [51]  3--33  9--33 15--33 16--33 19--33 21--33 23--33 24--33 30--33 31--33
## [61] 32--33  9--34 10--34 14--34 15--34 16--34 19--34 20--34 21--34 23--34
## [71] 24--34 27--34 28--34 29--34 30--34 31--34 32--34 33--34
```

- $n = |V|$: number of vertices, $m = |E|$: number of edges
- d_v: degree of vertex v (how many neighbours?)

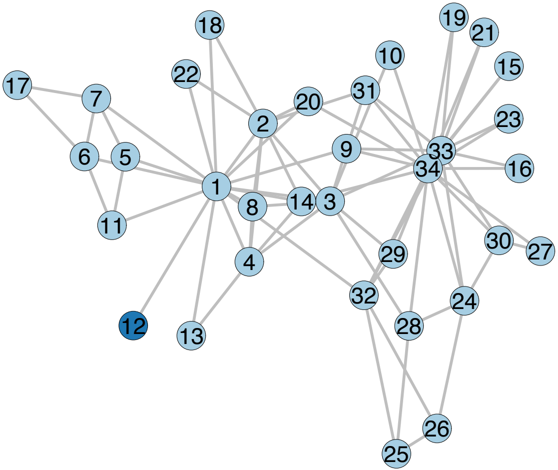
A classic example: Karate club (Zachary 1977)

- 34 members, edges = friendships
- The club split into two factions after a dispute
- Can we recover this split from the network alone?
- Who are the “influencers”?
  - **1**: the instructor (“Mr. Hi”)
  - **34**: the president (John A.)



Idea: split the network by cutting edges

- Consider two groups — find a set of edges to cut so the graph splits into two disconnected components
- **Min-cut**: the fewest edges to remove



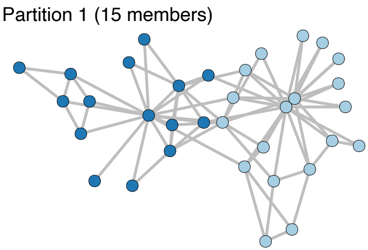
A trivial solution?

Split G by the fewest edges between two parties

```
.cuts <-
    as.directed(karate) %>%
    st_min_cuts(source = "1", target = c("33","34"))
.cuts$value / 2  # number of edges to cut
```

```
## [1] 5
```

Min-cut partitions from source to target



- Multiple solutions with the same min-cut $= 5$ edges
- But min-cut requires knowing **source and target** — can we find groups without labels?

The network as an adjacency matrix A

- Row/column: a member
- Non-zero element: friendship
- Can you see group structures?

$$A_{ij} = \begin{cases} 1 & i,j \text{ connected} \\ 0 & \text{otherwise} \end{cases}$$

Stochastic block model: a generative model of A?

- An adjacency matrix A can be **data**

$$A_{ij} \sim \text{Bernoulli}\left(\sum_{k,k'} \theta_{kk'} Z_{ik} Z_{jk}\right)$$

where

$$Z_{ik} = \begin{cases} 1 & i \in \text{ cluster/block/membership } k \\ 0 & \text{otherwise} \end{cases}$$

and

$$\theta_{kk'} = P(A = 1 | \text{between communities } k \text{ and } k')$$

- **Goal**: factorize $A \approx Z^{\top} \Theta Z$

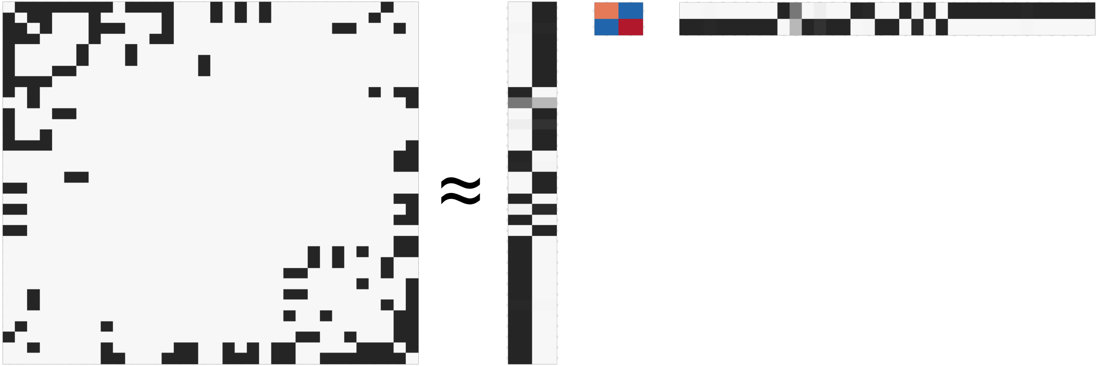
Fitting the stochastic block model

```r
library(hsblock)
.out <- fit.hsblock(A, "bernoulli", vbiter=100,
                    inner.iter=10, verbose=FALSE)
Z <- as.matrix(.out$Z)
```

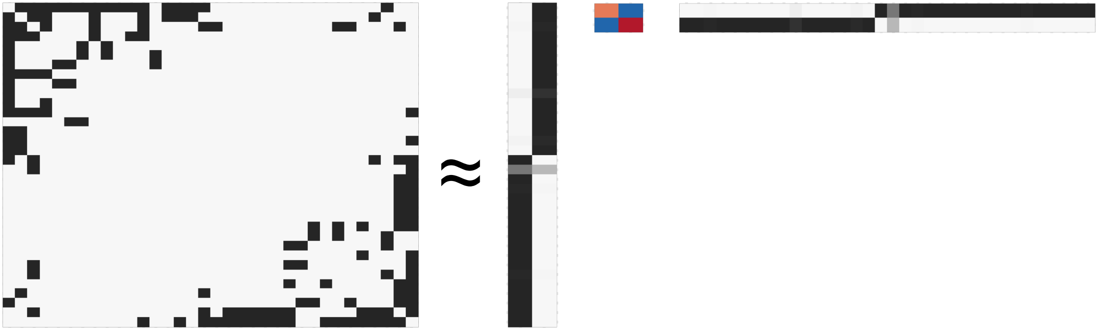
```r
n <- ncol(Z)
Edges <- 0.5 * Z %*% A %*% t(Z)
.ones <- matrix(1,n,n)
diag(.ones) <- 0
Tot <- Z %*% .ones %*% t(Z)/2
Theta <- Edges / Tot
```

- $\mathbf{Z}$: 2 clusters $\times$ 34 vertices
- Θ: edge probabilities

Matrix factorization view: $A \approx Z^\top \Theta Z$

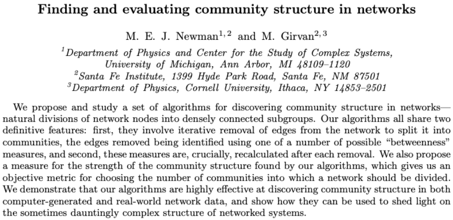


Reordering by cluster reveals the block structure



- Sorting vertices by cluster membership makes the block structure visible
- Each block shows edge density within/between groups

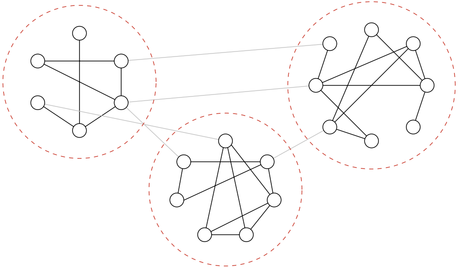
How do we know clustering is good enough?


Finding and evaluating community structure in networks

M. E. J. Newman[1,2] and M. Girvan[2,3]

[1] *Department of Physics and Center for the Study of Complex Systems, University of Michigan, Ann Arbor, MI 48109–1120*
[2] *Santa Fe Institute, 1399 Hyde Park Road, Santa Fe, NM 87501*
[3] *Department of Physics, Cornell University, Ithaca, NY 14853–2501*

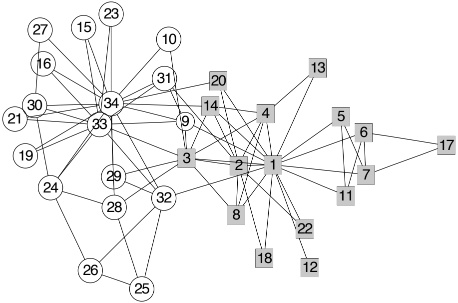
We propose and study a set of algorithms for discovering community structure in networks—natural divisions of network nodes into densely connected subgroups. Our algorithms all share two definitive features: first, they involve iterative removal of edges from the network to split it into communities, the edges removed being identified using one of a number of possible “betweenness” measures, and second, these measures are, crucially, recalculated after each removal. We also propose a measure for the strength of the community structure found by our algorithms, which gives us an objective metric for choosing the number of communities into which a network should be divided. We demonstrate that our algorithms are highly effective at discovering community structure in both computer-generated and real-world network data, and show how they can be used to shed light on the sometimes dauntingly complex structure of networked systems.



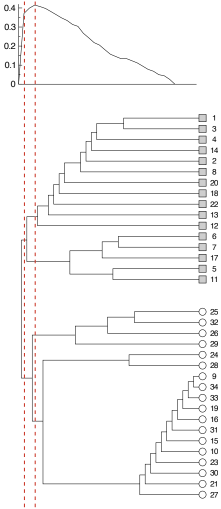
It’s not always as perfect as this. . .

- cluster ≈ densely connected subgraph
- but how dense?

In a dendrogram of clusters, where should we cut?



This is perhaps what happened after the split...



- **What will be the proper score?**
- The underlying combinatorial optimization is one of the hardest CS problems.

What is modularity? (Newman and Girvan 2004)

Given a partition of vertices into groups, **modularity** Q measures how many more edges fall *within* groups than expected by chance:

$$Q = \frac{1}{2m} \sum_{ij} \left[\mathbf{A}_{ij} - \frac{d_i d_j}{2m} \right] \delta(c_i, c_j)$$

- A_{ij}: adjacency matrix, d_i: degree of vertex i, m: total edges
- $\delta(c_i, c_j) = 1$ if vertices i, j are in the same group
- $Q \in [-0.5, 1]$; higher $=$ stronger community structure
- The null model: edges are distributed proportionally to degree

Why $d_i d_j / 2m$?

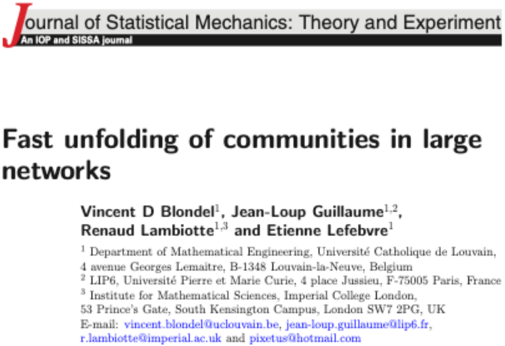
- Each vertex i has d_i "stubs" (half-edges). Total stubs: $\sum_i d_i = 2m$.
- Pick a stub from i. It connects to one of $2m - 1 \approx 2m$ other stubs.
- Vertex j has d_j stubs, so:

$$Pr(\text{stub from } i \text{ lands on } j) = \frac{d_j}{2m}$$

- Vertex i has d_i stubs, so the expected number of edges between i and j:

$$\mathbb{E}[A_{ij}] = \frac{d_i\, d_j}{2m}$$

Louvain: Greedy optimization of the modularity score



Journal of Statistical Mechanics: Theory and Experiment

An IOP and SISSA journal

Fast unfolding of communities in large networks

Vincent D Blondel[1], Jean-Loup Guillaume[1,2], Renaud Lambiotte[1,3] and Etienne Lefebvre[1]

[1] Department of Mathematical Engineering, Université Catholique de Louvain, 4 avenue Georges Lemaitre, B-1348 Louvain-la-Neuve, Belgium

[2] LIP6, Université Pierre et Marie Curie, 4 place Jussieu, F-75005 Paris, France

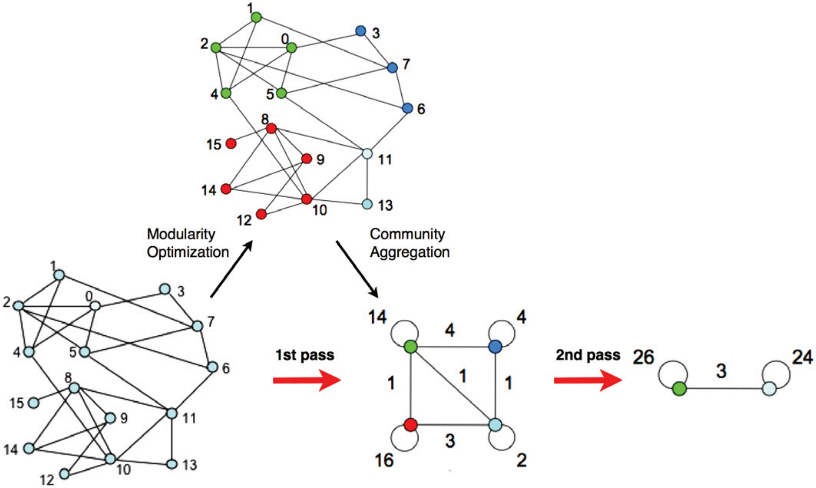
[3] Institute for Mathematical Sciences, Imperial College London, 53 Prince's Gate, South Kensington Campus, London SW7 2PG, UK

E-mail: vincent.blondel@uclouvain.be, jean-loup.guillaume@lip6.fr, r.lambiotte@imperial.ac.uk and pixetus@hotmail.com

(Blondel et al. 2008):

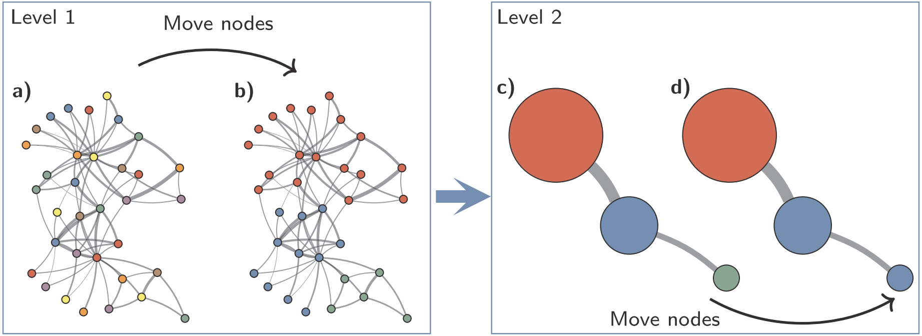
1. Each vertex starts as its own community
2. Move vertices to neighbour's community if it increases Q
3. Aggregate communities into super-nodes
4. Repeat until Q converges

Louvain algorithm: vertices $\rightarrow$ “super” nodes



(Blondel et al. 2008)

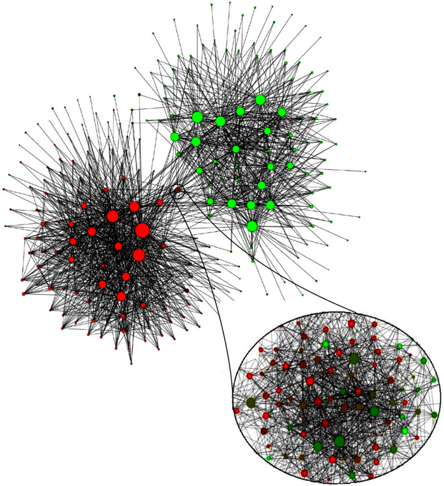
Louvain deals with the “colours” of “super” nodes



Key idea: Keep coarsening a large graph into a smaller one

(Traag et al. 2019)

Louvain can easily scale up to a million vertices



(Blondel et al. 2008)

Louvain's supernode with disconnected components


SCIENTIFIC REPORTS

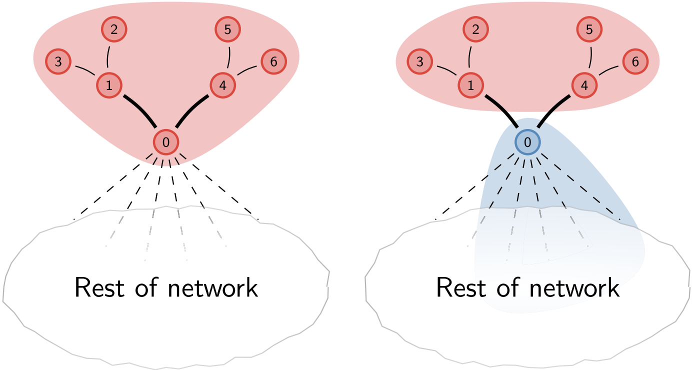
OPEN

From Louvain to Leiden: guaranteeing well-connected communities

Received: 20 November 2018
Accepted: 11 March 2019
Published online: 26 March 2019

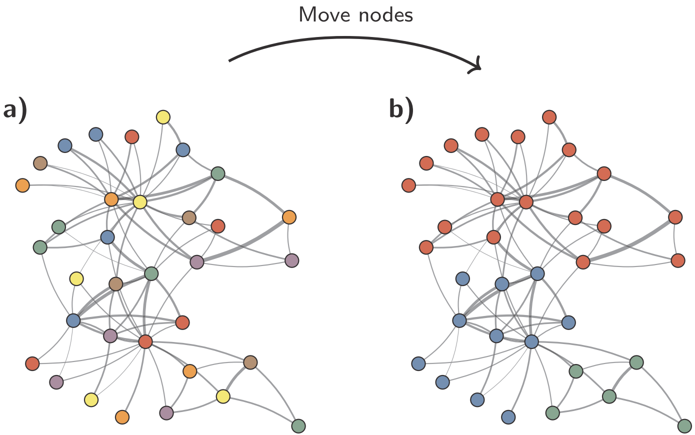
V. A. Traag, L. Waltman & N. J. van Eck

Community detection is often used to understand the structure of large and complex networks. One of



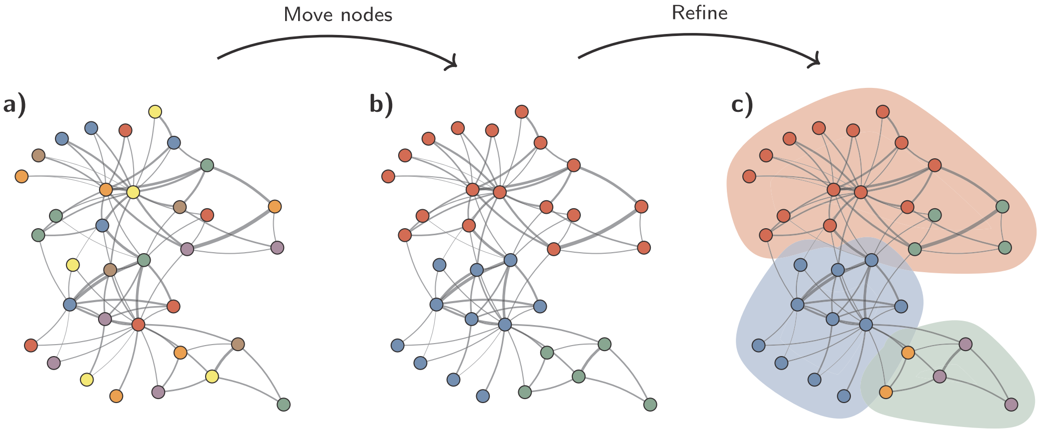
(Traag et al. 2019)

Leiden “refines” to ensure connected within a cluster



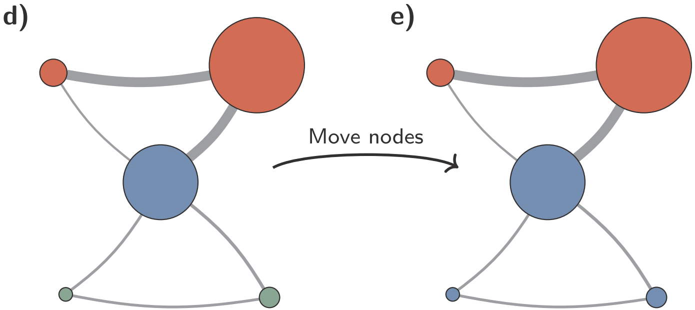
(Traag et al. 2019)

Leiden "refines" to ensure connected within a cluster



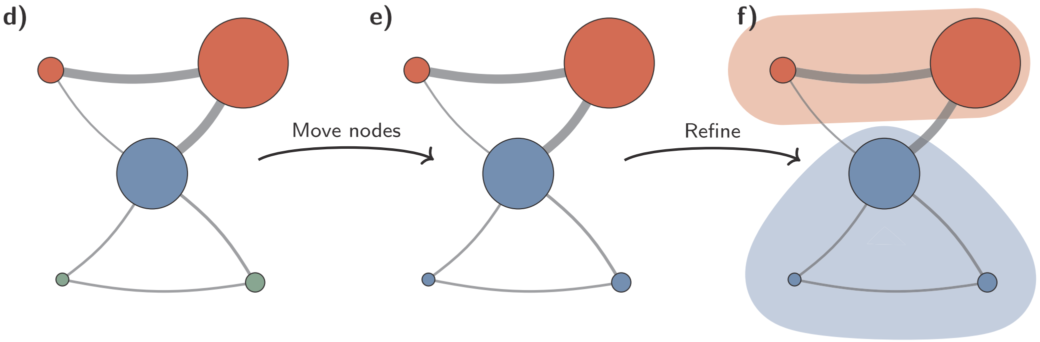
(Traag et al. 2019)

Leiden "refines" to ensure connected within a cluster



(Traag et al. 2019)

Leiden "refines" to ensure connected within a cluster

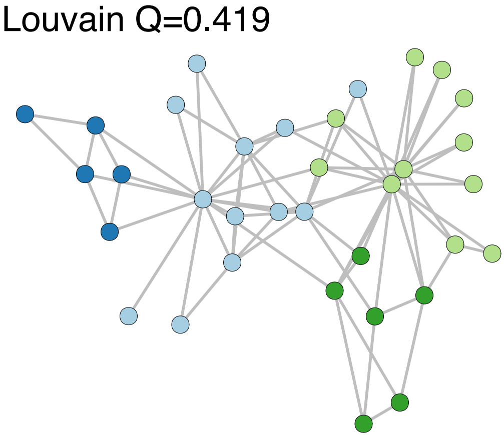


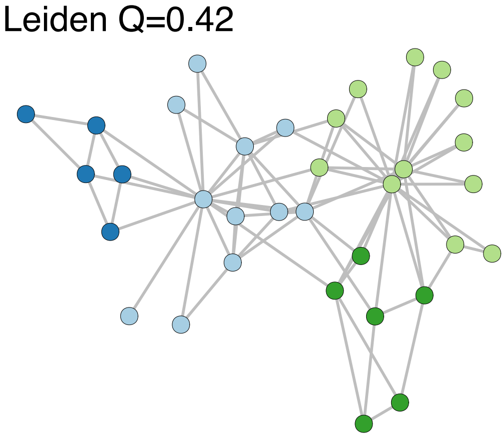
(Traag et al. 2019)

Leiden vs. Louvain

```r
cl.louvain <- cluster_louvain(karate)
cl.leiden  <- cluster_leiden(karate,
    objective_function = "modularity")
```

Leiden vs. Louvain



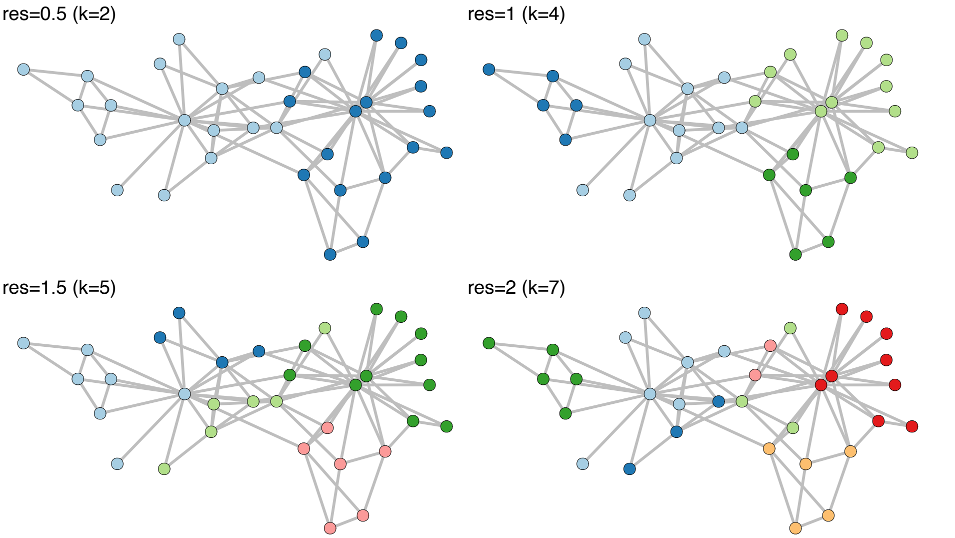


What should we learn from Louvain/Leiden?

When n is large. . .

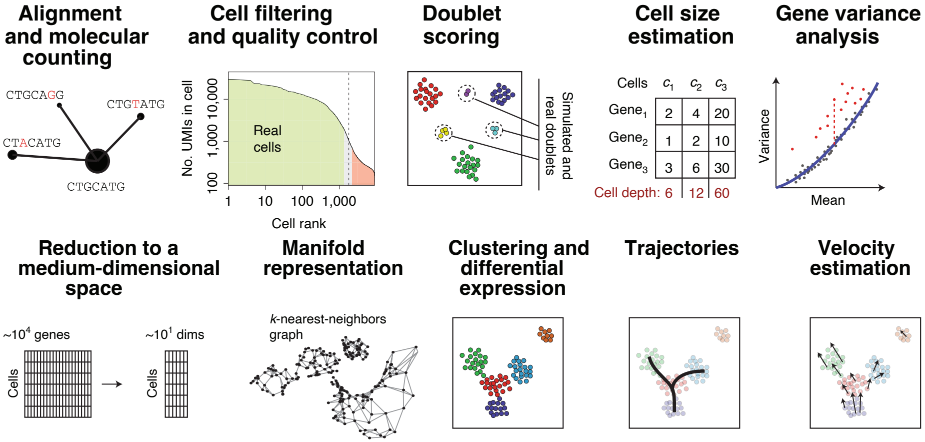
- What shall we do?
- Any counter examples for Leiden?

Side note: The resolution parameter matters



Today's lecture

Recall: the standard single-cell workflow



(Kharchenko 2021)

Clustering the PDAC data

```r
pdac <- Read10X(data.dir = data.dir)
pdac <- CreateSeuratObject(counts = pdac,
    project = "PDAC_TISSUE_1",
    min.cells = 3, min.features = 200)

pdac <- pdac |>
    NormalizeData() |>
    FindVariableFeatures(nfeatures = 2000) |>
    ScaleData() |>
    RunPCA(npcs = 30) |>
    FindNeighbors(dims = 1:20) |>
    RunUMAP(dims = 1:20)
```

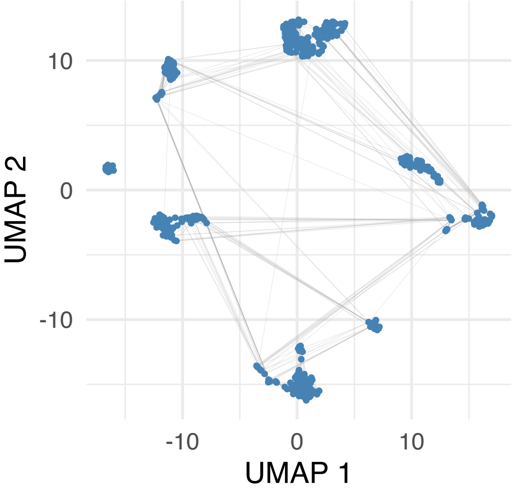
- Same PDAC Tissue 1 dataset from Lecture 8 (Steele et al. 2020)
- `FindNeighbors()` builds the **KNN** and **SNN** graphs in PCA space

Building the KNN graph

```r
knn.mat <- pdac@graphs$RNA_nn
dim(knn.mat)
```

```
## [1] 995 995
```

- `FindNeighbors(dims = 1:20)` connects each cell to its $k = 20$ nearest neighbours in PCA space
- $A_{ij} = 1$ if cell j is among k-NN of cell i

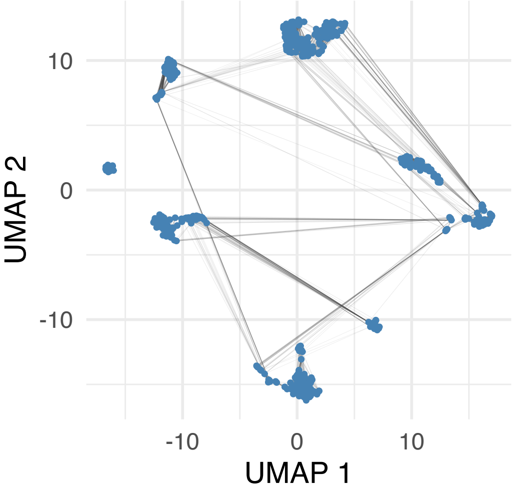


Shared neighbours graph

- **SNN**: weight edges by shared neighbour overlap (Jaccard index)

$$w_{ij} = \frac{|N_i \cap N_j|}{|N_i \cup N_j|}$$

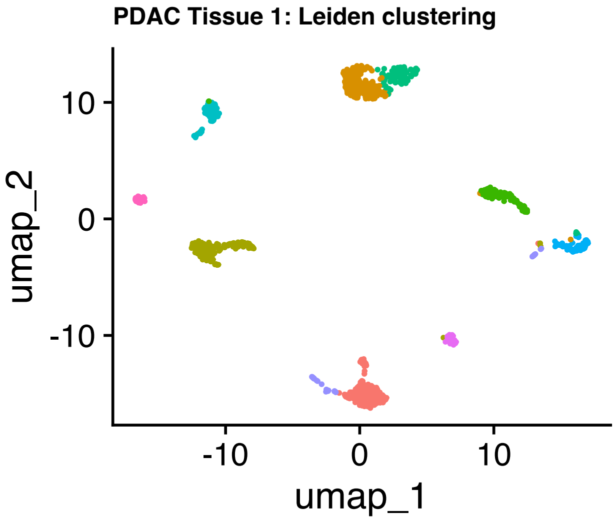
- Cells sharing many neighbours get stronger connections
- This is what Leiden/Louvain clusters on



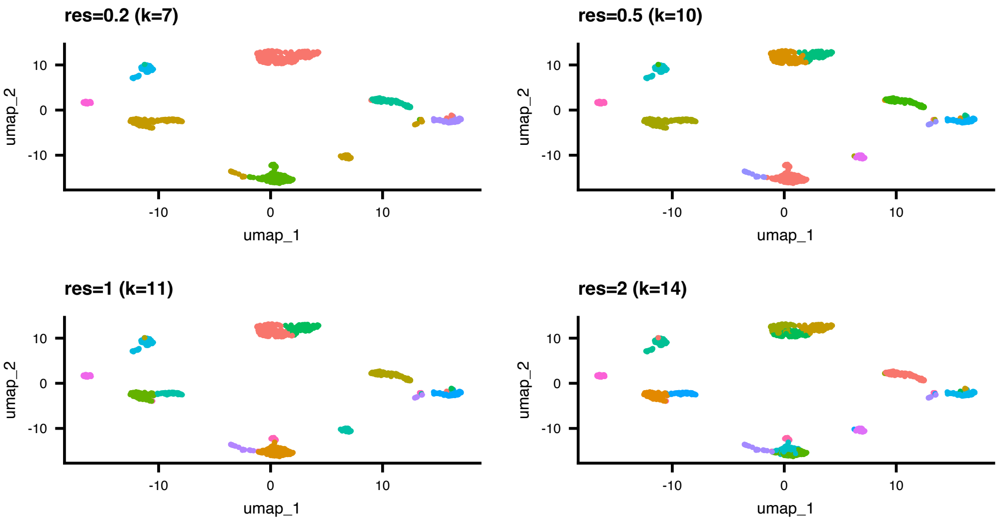
Clustering the PDAC data

```r
pdac <- FindClusters(
    pdac,
    algorithm = 4,
    resolution = 0.5)
```

- Leiden on the SNN graph
- 10 clusters found
- Resolution = 0.5



Effect of the resolution parameter



The resolution parameter in the modularity score

The **resolution parameter** γ scales the null model:

$$Q_\gamma = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \gamma \frac{d_i d_j}{2m} \right] \delta(c_i, c_j)$$

- $\gamma > 1$: penalizes large clusters more $\rightarrow$ **more, smaller** clusters
- $\gamma < 1$: tolerates sparser communities $\rightarrow$ **fewer, larger** clusters
- $\gamma = 1$: standard modularity

Today's lecture

1. Graph-based clustering
2. Single-cell clustering in practice
3. When clustering goes wrong
4. Discussion: How should we test cell-cell communication?
5. Appendix

Are clusters real or artifacts of the algorithm?

nature methods

Article https://doi.org/10.1038/s41592-023-01933-9

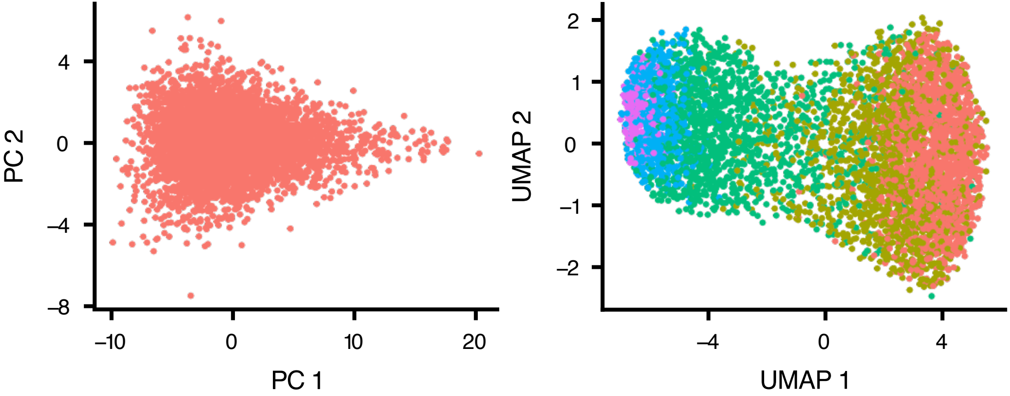
Significance analysis for clustering with single-cell RNA-sequencing data

Received: 1 August 2022

Isabella N. Grabski [1], Kelly Street [2] & Rafael A. Irizarry [3]

(Grabski et al. 2023)

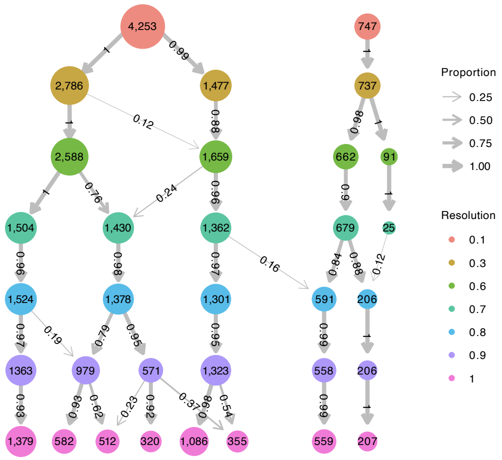
Clustering a single population finds spurious clusters



Simulated data with **no true subpopulations**. PCA (left) shows one blob; Leiden (right) still finds clusters.

(Grabski et al. 2023)

Higher resolution keeps splitting, but are they real?



Each node = a cluster at a given resolution. Arrows show how cells redistribute as resolution increases.

Thin arrows = small proportion of cells moving → potentially spurious splits.

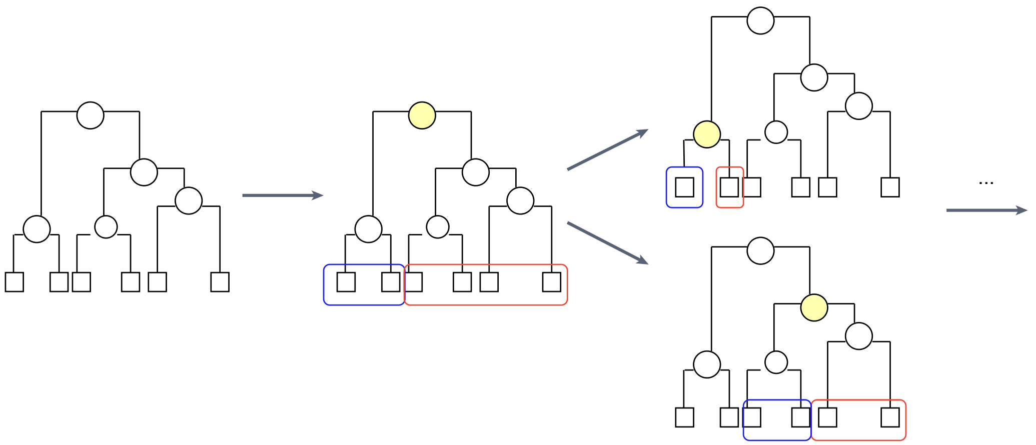
(Grabski et al. 2023)

sc-SHC: recursively test each split for significance

Hierarchically cluster cells (or pre-computed clusters)

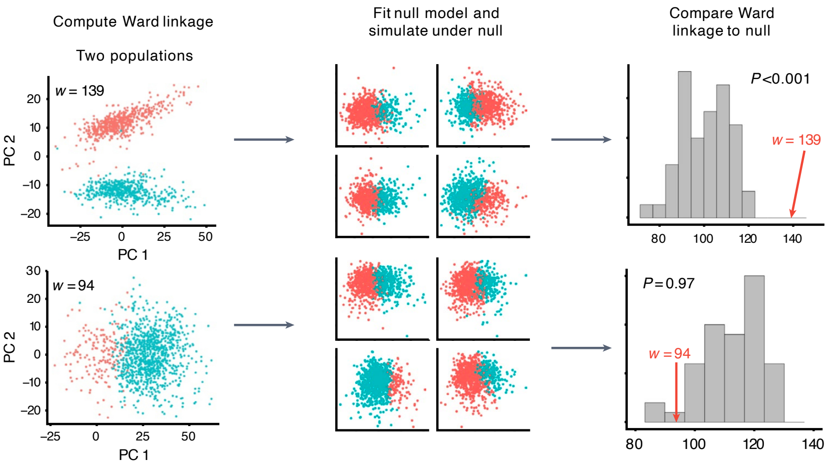
Test split at the root

Stop if fail to reject; otherwise proceed recursively



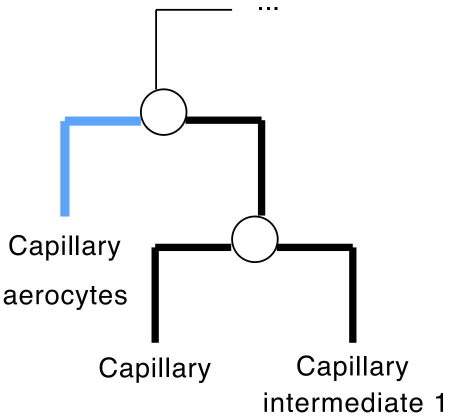
(Grabski et al. 2023)

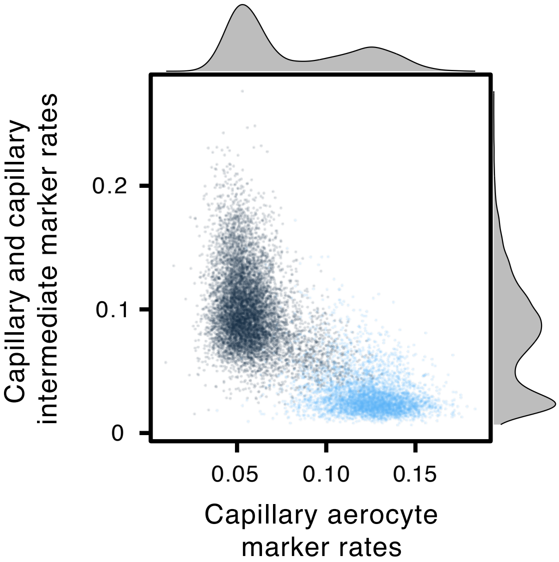
Can we compute the p-value of each split?



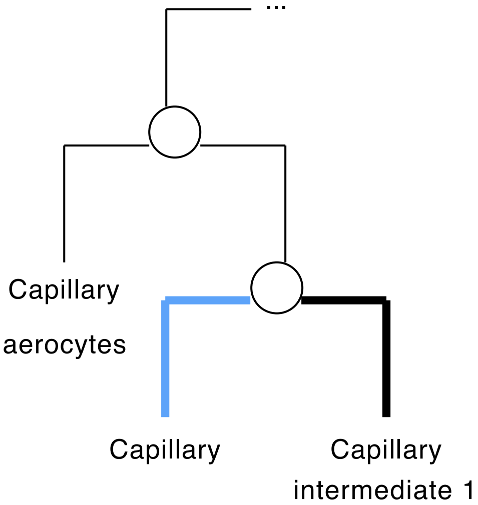
Top: two real populations → observed w far from null ($P < 0.001$). Bottom: one population → w typical under null ($P = 0.97$). (Grabski et al. 2023)

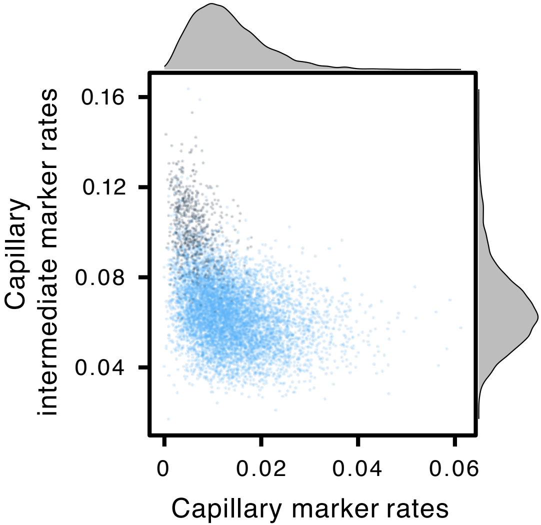
Capillary aerocytes are a distinct subtype





Not all splits are real: capillary subtypes overlap





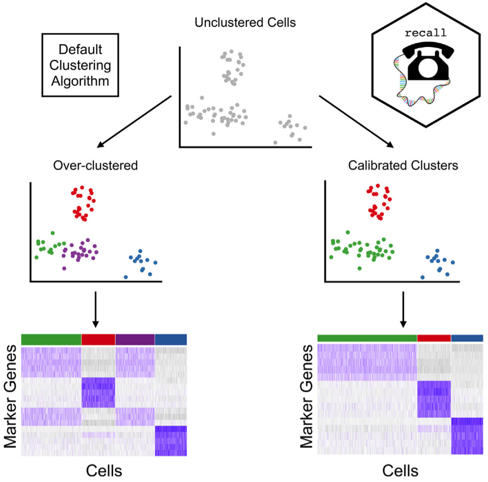
RECALL combats against an over-clustering problem

Problem: Leiden with a high resolution parameter → too many clusters → spurious DE genes

RECALL (calibrated clustering with artificial variables):

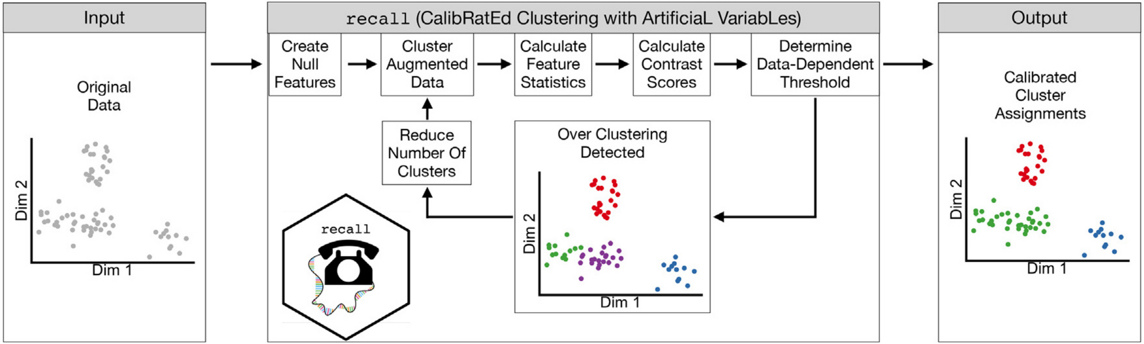
1. **Augment** the expression matrix with **artificial null genes** — synthetic variables that match the data distribution but have no association with cell types
2. **Cluster** cells using both real and artificial genes
3. **Test for DE** between clusters, for both real and artificial genes
4. Since artificial genes are known nulls, their test statistics reveal the **double-dipping effect**
5. If real genes are not more significant than artificial ones → **merge** those clusters
6. Repeat until clusters are well-calibrated

Default clustering over-clusters

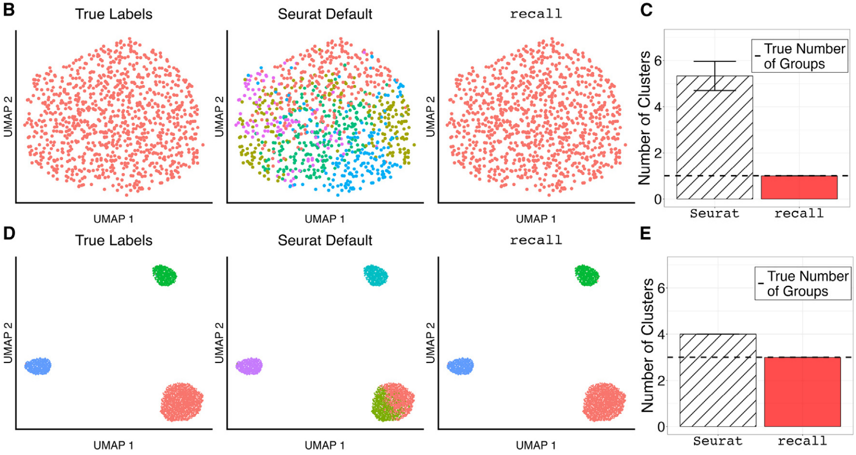


(DenAdel et al. 2025)

RECALL: iteratively reduce resolution until calibrated

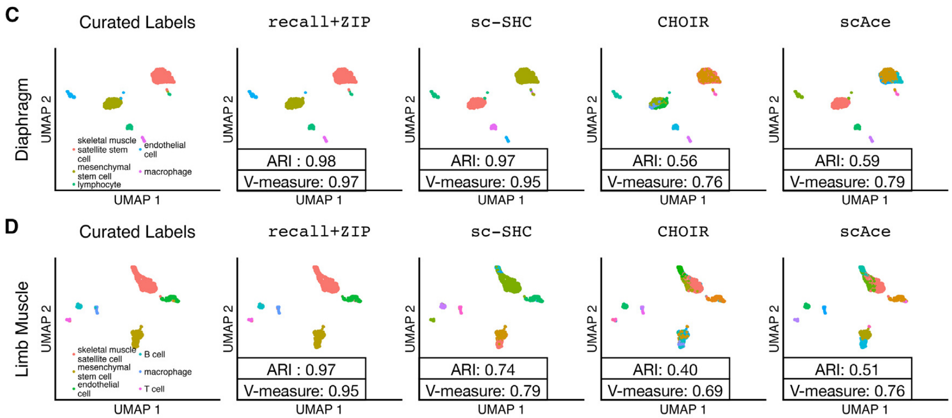


RECALL correctly identifies true clusters



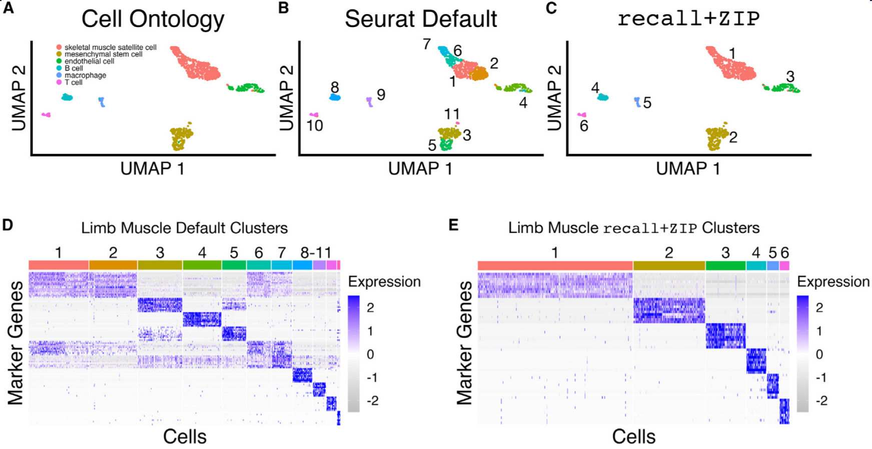
B–C: one population — Seurat finds about 5, RECALL finds 1. D–E: three populations — both recover 3. (DenAdel et al. 2025)

RECALL outperforms alternatives on real tissue data



Diaphragm and limb muscle datasets. ARI and V-measure vs curated labels. (DenAdel et al. 2025)

Seurat default over-splits; RECALL matches cell ontology



Limb muscle: Seurat finds 11 clusters, RECALL finds 6 matching the 6 annotated cell types. Heatmaps show cleaner marker gene blocks. (DenAdel et al. 2025)

How does RECALL generate artificial null genes?

For each gene j, fit a **zero-inflated Poisson** (ZIP) to the observed counts:

$$P(X_j = 0) = \pi_{0j} + (1 - \pi_{0j})\, e^{-\lambda_j}, \quad P(X_j = x) = (1 - \pi_{0j})\, \frac{\lambda_j^x\, e^{-\lambda_j}}{x!}$$

Estimate parameters from the data:

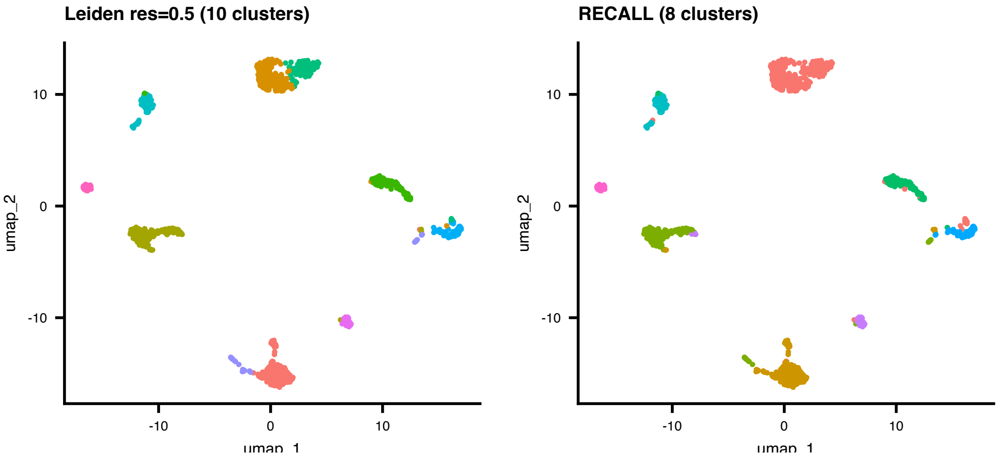
- $\hat{\lambda}_j$ from the mean $\bar{x}_j$ and zero-fraction r_{0j} via the Lambert W function
- $\hat{\pi}_{0j} = 1 - \bar{x}_j / \hat{\lambda}_j$

Then sample synthetic null genes: $\tilde{x}_j \sim \mathrm{ZIP}(\hat{\pi}_{0j},\, \hat{\lambda}_j)$

Each null gene matches the **marginal** distribution (mean, zero-inflation) of the real gene.

RECALL on the PDAC data

```r
library(recall)
pdac.recall <- FindClustersRecall(pdac,
    algorithm = "leiden",
    resolution_start = 0.5, dims = 1:20)
```

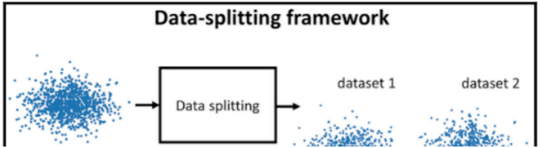


Cell Systems

Report

Valid Post-clustering Differential Analysis for Single-Cell RNA-Seq

Graphical Abstract



Authors

Jesse M. Zhang, Govinda M. Kamath, David N. Tse

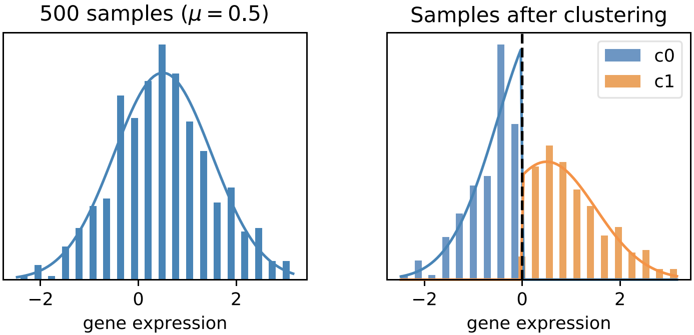
Correspondence

dntse@stanford.edu

- Cluster cells using gene expression $\rightarrow$ find marker genes that differ between clusters
- But clusters were *defined* to maximize gene expression differences
- This is **selection bias**: the same data used to form and evaluate the hypothesis

(Zhang et al. 2019)

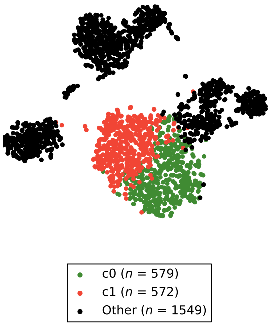
Clustering creates artificial DE even from one population



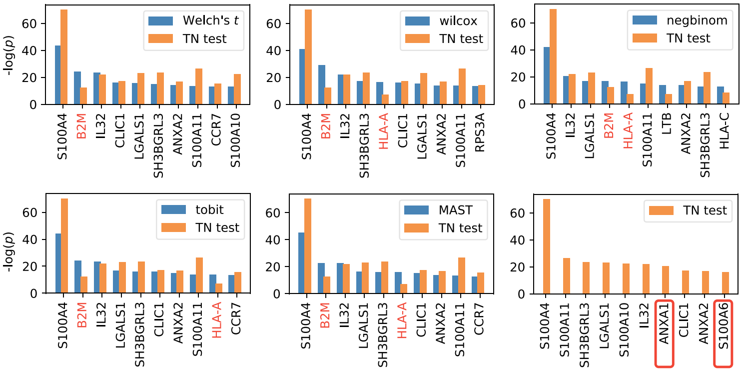
500 cells from **one** distribution ($\mu = 0.5$). After k-means, the two clusters look different — but the difference is an artefact. (Zhang et al. 2019)

Standard DE tests are inflated; corrected tests are not

10x PBMC dataset



Most significant genes for Seurat differential expression tests (c0 v c1)



Blue = standard test, orange = truncated normal (TN) corrected test. Red gene names = housekeeping genes that should not be DE. (Zhang et al. 2019)

Clustering then testing $\rightarrow$ inflated significance

Even when there is **no true grouping**, standard DE after clustering produces many "significant" genes:

- A "double dipping" problem
- The clustering algorithm forces cells into groups
- Cells assigned to the same cluster are more similar *by construction*
- Standard tests (e.g., Wilcoxon, t-test) ignore this selection step $\rightarrow$ **inflated p-values**

The double-dipping problem

We observe a count matrix X (cells $\times$ genes), where $X_{ij} \sim \text{Poisson}(\lambda_{ij})$

Standard pipeline (double dipping):

- Use X to estimate clusters $\hat{c}_1, \ldots, \hat{c}_n$
- Use X again to test: "Is gene j differentially expressed between clusters?"
- The test does not account for the fact that clusters were chosen to make genes look different

Result: under the null (λ_{ij} does not depend on cell type), we still get many small p-values

(Neufeld et al. 2023)

Inference after latent variable estimation for single-cell RNA sequencing data

ANNA NEUFELD*

Department of Statistics, University of Washington, Seattle, WA 98195, USA

aneufeld@uw.edu

LUCY L. GAO

Department of Statistics, University of British Columbia, BC V6T 1Z4, Canada

JOSHUA POPP

Department of Biomedical Engineering, Johns Hopkins University, Baltimore, MD 21218, USA

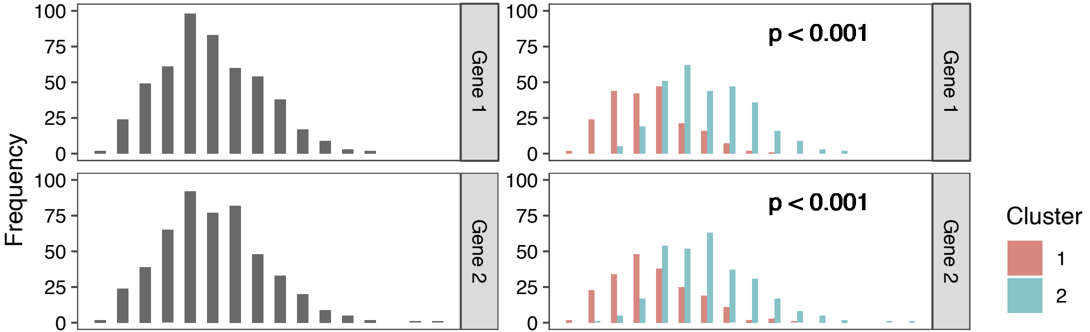
ALEXIS BATTLE

Department of Biomedical Engineering, Johns Hopkins University, Baltimore, MD 21218, USA and Department of Computer Science, Johns Hopkins University, Baltimore, MD 21218, USA

DANIELA WITTEN

Department of Statistics, University of Washington, Seattle, WA 98195, USA and Department of Biostatistics, University of Washington, Seattle, WA 98195, USA

Null genes look DE after clustering: both $p < 0.001$



Two genes drawn from the **same** Poisson distribution across all cells. After clustering (right), both appear significantly DE ($p < 0.001$). (Neufeld et al. 2023)

Count splitting (Neufeld et al. 2023)

Key insight: Poisson thinning creates independent splits

If $X_{ij} \sim \text{Poisson}(\lambda_{ij})$, then for any $\epsilon \in (0, 1)$:

$$X_{ij}^{\text{train}} \mid X_{ij} \sim \text{Binomial}(X_{ij}, \epsilon), \quad X_{ij}^{\text{test}} = X_{ij} - X_{ij}^{\text{train}}$$

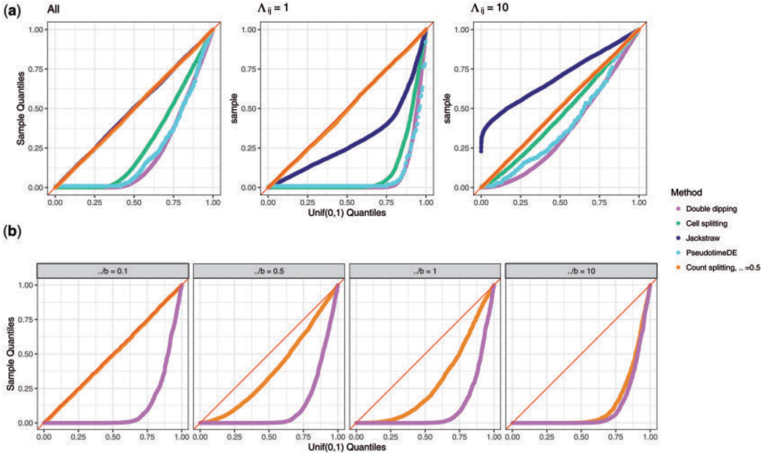
Then $X^{\text{train}} \perp X^{\text{test}}$ (they are **independent**!)

The recipe:

1. Split X into X^{train} and X^{test} via Poisson thinning
2. Cluster using X^{train} only
3. Test for DE using X^{test} only

This controls Type I error because the test data is independent of the clustering.

Count splitting controls Type I error

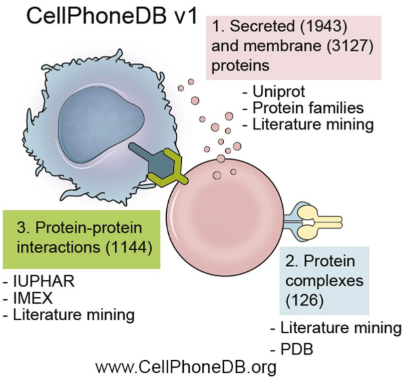


- QQ plots under null: double dipping (purple) inflates p-values
- Count splitting (orange) stays on the diagonal.
- Count splitting across signal-to-noise ratios.

(Neufeld et al. 2023)

Today's lecture

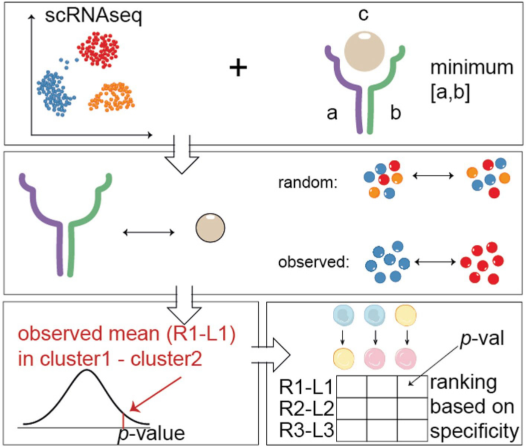
CellPhoneDB infers communication from clusters



1. **Cluster** cells into discrete types (e.g., Leiden)
2. **Look up** known ligand-receptor (L-R) pairs from a curated database
3. For each L-R pair (l, r) and each cell-type pair (A, B):
   - Compute the **mean expression** of l in A and r in B
   - Score $= \text{mean}(\bar{x}_l^A, \bar{x}_r^B)$
4. **Permutation test**: randomly shuffle cell-type labels $\rightarrow$ null distribution of scores
5. Report pairs with $p < 0.05$

(Vento-Tormo et al. 2018; Armingol et al. 2021; Efremova et al. 2020)

CellPhoneDB output: significant L-R pairs



(Vento-Tormo et al. 2018)

Discussion

Given what we learned today, what could go wrong with this approach?

Hint 1: Clustering uses the same expression matrix as . . .

Hint 2: But L-R genes may not overlap with the genes that . . .

Discussion

What will be the underlying generative model?

Can we fix this? Could count splitting help?

- Split counts: $X = X^{\text{train}} + X^{\text{test}}$
- Cluster on X^{train}, test L-R pairs on X^{test}

But wait ...

- L-R scores need **mean expression** — do we have enough signal in X^{test}?
- Ligand in cluster A, receptor in cluster B — we need counts from **different cells**, not the same cell split twice
- What if the clustering itself changes with only half the counts?

Takeaways

Graph-based clustering

- Modularity optimization (Louvain/Leiden) is the workhorse of single-cell clustering
- The resolution parameter controls granularity — there is no single “correct” value
- SBMs offer a principled alternative with automatic model selection

When clustering goes wrong

- Clustering then testing on the same data = **double dipping** $\rightarrow$ inflated p-values
- **sc-SHC**: build significance testing into the clustering itself
- **Count splitting**: exploit Poisson thinning to create independent train/test sets
- **RECALL**: use artificial null genes to calibrate and detect over-clustering

Today's lecture

No perfect clustering algorithm

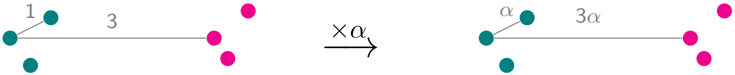
Impossibility Theorem (Kleinberg 2002): no clustering function f can simultaneously satisfy all three axioms:

1. **Scale invariance**: scaling all distances by $\alpha > 0$ doesn't change the clustering
2. **Richness**: every possible partition can be produced by some distance function
3. **Consistency**: shrinking within-cluster distances and expanding between-cluster distances doesn't change the clustering

Any algorithm must violate at least one:

Method	Scale inv.	Richness	Consistency
k-cluster (fixed k)	✓	×	✓
Distance threshold (fixed r)	×	✓	✓
Proportional ($\alpha \cdot d_{\max}$)	✓	✓	×

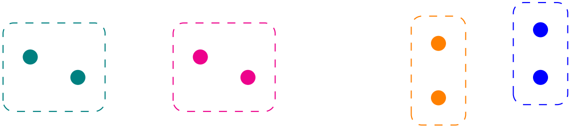
Axiom 1: Scale invariance



$f(d) = \{A, B\}$ $\qquad$ $f(\alpha \cdot d) = \{A, B\}$

Multiplying all distances by $\alpha > 0$ should not change the partition.

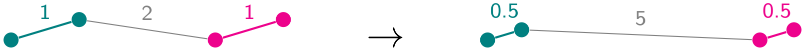
Axiom 2: Richness



$d_1 \to \{AB, CD\}$ $\qquad$ $d_2 \to \{AC, BD\}$

For **any** partition of n points, there exists some distance function d that produces it.

Axiom 3: Consistency



$f(d) = \{A, B\}$ $\qquad$ $f(d') = \{A, B\}$?

If we make clusters “tighter” and “farther apart,” the same partition should remain.

References

Armingol, Erick, Adam Officer, Olivier Harismendy, and Nathan E Lewis. 2021. "Deciphering Cell-Cell Interactions and Communication from Gene Expression." *Nat. Rev. Genet.* 22 (2): 71–88.

Blondel, Vincent D, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. "Fast Unfolding of Communities in Large Networks." *Journal of Statistical Mechanics: Theory and Experiment*.

DenAdel, Alan, Michelle L Ramseier, Andrew W Navia, et al. 2025. "Artificial Variables Help to Avoid over-Clustering in Single-Cell RNA Sequencing." *Am. J. Hum. Genet.* 112 (4): 940–51.

References

Efremova, Mirjana, Miquel Vento-Tormo, Sarah A Teichmann, and Roser Vento-Tormo. 2020. "CellPhoneDB: Inferring Cell-Cell Communication from Combined Expression of Multi-Subunit Ligand-Receptor Complexes." *Nat. Protoc.* 15 (4): 1484–506.

Grabski, Isabella N, Kelly Street, and Rafael A Irizarry. 2023. "Significance Analysis for Clustering with Single-Cell RNA-Sequencing Data." *Nat. Methods*, July.

Kharchenko, Peter V. 2021. "The Triumphs and Limitations of Computational Methods for scRNA-Seq." *Nat. Methods*, June.

Kleinberg, Jon. 2002. "An Impossibility Theorem for Clustering." *Advances in Neural Information Processing Systems* 15.

References

Neufeld, Anna, Lucy L Gao, Joshua Popp, Alexis Battle, and Daniela Witten. 2023. "Inference After Latent Variable Estimation for Single-Cell RNA Sequencing Data." *Biostatistics* 25 (1): 270–87.

Newman, M E J, and M Girvan. 2004. "Finding and Evaluating Community Structure in Networks." *Phys. Rev. E Stat. Nonlin. Soft Matter Phys.* 69 (2 Pt 2): 026113.

Steele, Nina G, Eileen S Carpenter, Samantha B Kemp, et al. 2020. "Multimodal Mapping of the Tumor and Peripheral Blood Immune Landscape in Human Pancreatic Cancer." *Nature Cancer* 1 (11): 1097–112.

References

Traag, V A, L Waltman, and N J van Eck. 2019. "From Louvain to Leiden: Guaranteeing Well-Connected Communities." *Sci. Rep.* 9 (1): 5233.

Vento-Tormo, Roser, Mirjana Efremova, Rachel A Botting, et al. 2018. "Single-Cell Reconstruction of the Early Maternal-Fetal Interface in Humans." *Nature* 563 (7731): 347–53.

Zachary, Wayne W. 1977. "An Information Flow Model for Conflict and Fission in Small Groups." *Journal of Anthropological Research* 33 (4): 452–73.

References

Zhang, Jesse M, Govinda M Kamath, and David N Tse. 2019. "Valid Post-Clustering Differential Analysis for Single-Cell RNA-Seq." *Cell Systems*, Testing statistical hypotheses, vol. 9 (4): 383–392.e6.